

MR Users Manual

Editing, navigation, automation, and tools

Dr. Michael 'iDoc' Raus

5 August 2026

This manual explains the current MR application. Its sequence is inspired by the original Multi-Edit User's Guide, but every description has been reconsidered for the present Linux terminal editor. Historical names are used only where the modern feature still carries the same working idea.

Preface

MR is a terminal-native programmer's editor with a window system, a large document model, configurable editing profiles, and an in-memory macro language. The application deliberately does not make the user choose between the immediacy of a text-mode editor and the facilities normally associated with an IDE. Files, output, searches, workspaces, binary data, compiler diagnostics, and macros all live in the same TVision desktop.

The original Multi-Edit manual supplied a particularly practical order: begin with the screen and cursor, proceed through text and blocks, and only then introduce automation. This book follows that route without treating the historical program as a specification. DOS shells, printer drivers, page-break formatters, and other facilities that do not exist in MR have no chapter here. Their place is taken only where a current feature solves the same user problem.

The screenshots were taken from real MR sessions in a PTY. They are not mock-ups. The active colors, available profiles, filenames, and function-key labels therefore reflect an installed development configuration; another installation may use a different theme or keymap. The operation names and dialog structure are the stable part.

This is a task-oriented manual. The MRMAC Reference Manual remains the authority for language syntax and individual primitives, while the Technical Manual explains implementation boundaries and performance design.

Contents

Preface	ii
I The editing environment	1
1 Basic Concepts	2
1.1 The desktop, windows, and documents	2
1.2 Menus and the status line	3
1.3 Help and explanatory messages	3
1.4 Progress and performance observation	3
1.5 Editor modes	3
1.6 Fullscreen presentation	3
1.7 A productive first session	4
2 Cursor Control and Navigation	5
2.1 Character, line, and page motion	5
2.2 Going directly to a line	5
2.3 Position markers	5
2.4 Matching and structural motion	5
2.5 Mouse navigation	6
3 Installation and Setup	7
3.1 One application, staged configuration	7
3.2 Keymap profiles	7
3.3 Colors and themes	8
3.4 Paths	8
3.5 Backups and autosave	8
3.6 User-interface and live-log settings	8
4 Command-Line Use	9
4.1 Starting with files	9
4.2 Recursive loading	9
4.3 Help	9
4.4 Startup macro automation	9
4.5 Binary files at startup	10
5 Loading and Saving Files	11
5.1 Open, Load, and Get Last	11
5.2 Acquire, live logs, and the journal	11
5.3 Text and binary loading	11
5.4 Save, Save As, Save All, and Revert	12
5.5 File information	12
5.6 Workspaces	12
6 Inserting Text	13
6.1 Insert and overwrite	13
6.2 Lines and line endings	13
6.3 Clipboard input	13
6.4 Unicode input	13

6.5	Snippets and macro-generated text	13
7	Deleting Text	14
7.1	Character deletion	14
7.2	Selections and blocks	14
7.3	Deleting across windows	14
7.4	Recovering a deletion	14
8	Tabs and Virtual Space	15
8.1	Tab width and expansion policy	15
8.2	Typing beyond end of line	15
8.3	Tab-aware blocks	15
8.4	Indent and Undent	15
8.5	Format line and margins	15
9	Block Operations	16
9.1	Three block shapes	16
9.2	Marking state and cursor motion	16
9.3	Copy, move, and delete	17
9.4	Inter-window transfer	17
9.5	Indent, Undent, and transformations	17
9.6	Loading and saving blocks	17
10	Undo and Redo	18
10.1	Document-local history	18
10.2	Traversing the history	18
10.3	Undo and large files	18
10.4	Undo versus persistent recovery	18
10.5	Macros and undo	18
11	Windows, Virtual Desktops, and Workspaces	19
11.1	Creating, closing, and selecting windows	19
11.2	Hide, minimize, and zoom	19
11.3	Cascade, tile, and split	19
11.4	Virtual desktops	19
11.5	Links	19
11.6	Workspace lifecycle	20
12	File Formatting	21
12.1	Formatting scope	21
12.2	Paragraph reformatting	21
12.3	Code reformatting	21
12.4	Whitespace policy	21
12.5	Exporting to PDF	21
12.6	File comparison	22
13	The Format Line and Margins	23
13.1	Purpose	23
13.2	Reading the ruler	23
13.3	Left and right margins	23
13.4	Tabs and the format line	23
13.5	Profile inheritance	23
14	Search and Replace	24
14.1	The search dialog	24
14.2	Literal, word, and regular-expression searches	24
14.3	Replace	25
14.4	Repeating and continuing	25
14.5	Results and navigation	25
14.6	Multiple-file search	25
14.7	Multiple-file search and replace	25

15 Keystroke Macros	26
15.1 Recording	26
15.2 Naming and binding the result	26
15.3 The Macro Library	26
15.4 What recording does not capture	26
15.5 Saving and reloading	26
16 Line Drawing	27
16.1 Enabling the mode	27
16.2 Drawing with cursor motion	27
16.3 Drawing with the mouse	27
16.4 Rectangles from column blocks	27
16.5 Single and double intersections	27
17 Calculator	28
17.1 Opening the calculator	28
17.2 Entering values	28
17.3 Arithmetic and logical operations	29
17.4 Memory and pasting the result	29
18 Character Tables	30
18.1 ASCII and Latin-1	30
18.2 Emoji and Unicode symbols	30
19 Dense Display and Information Projection	32
19.1 Keeping document space visible	32
19.2 Gutters	32
19.3 The minimap	32
19.4 Folding and zoom	32
19.5 Terminal size and remote sessions	33
20 Filename-Extension Profiles	34
20.1 Why profiles exist	34
20.2 Identity and matching	34
20.3 Language and editor behavior	35
20.4 Build and lifecycle integration	35
20.5 Managing profile drafts	35
21 Indenting Styles	36
21.1 Automatic indentation	36
21.2 Tabs, spaces, and visual columns	36
21.3 Indenting and undenting blocks	36
21.4 Language-sensitive structure	36
22 Language-Specific Features	37
22.1 Detection and syntax projection	37
22.2 Code folding and outlines	37
22.3 Parentheses and structural navigation	38
22.4 Formatting source code	38
22.5 Compiler profiles and diagnostics	38
22.6 Git status and file changes	38
22.7 File comparison	38
23 The MRMAC Interface	40
23.1 What MRMAC contributes	40
23.2 Macro files and definitions	40
23.3 Execution contexts	40
23.4 Menus, dialogs, and modeless tools	40
23.5 Keys, recording, and AUTOEXEC	40
23.6 Running a macro from the command line	41
23.7 Debugging and reference material	41

Part I

The editing environment

Chapter 1

Basic Concepts

1.1 The desktop, windows, and documents

MR starts inside a terminal but does not behave like a single-buffer full-screen program. TVision supplies a desktop on which editor windows, Bento workspaces, modeless macro tools, Help, and modal dialogs coexist. A window is the visible and interactive container; a document is the text or binary content behind an editor. More than one viewport may refer to the same document, and a workspace may restore several different documents and their geometry.

The active window has a highlighted frame. Its title identifies the persistent file when one exists. Frame markers report properties such as modification, Git changes, read-only state, insert mode, language, workspace-main status, macro recording, or background work. Hovering over an ordinary marker explains it. Hovering over the activity marker opens a transient list of that window's running work; clicking the marker pins the list until it is clicked again, closed, or dismissed with **Esc**. The list observes work but does not cancel it. Line and column information appears in the configured cursor-position marker. The editor's left and right projections can contain line numbers, fold markers, difference markers, or a minimap.

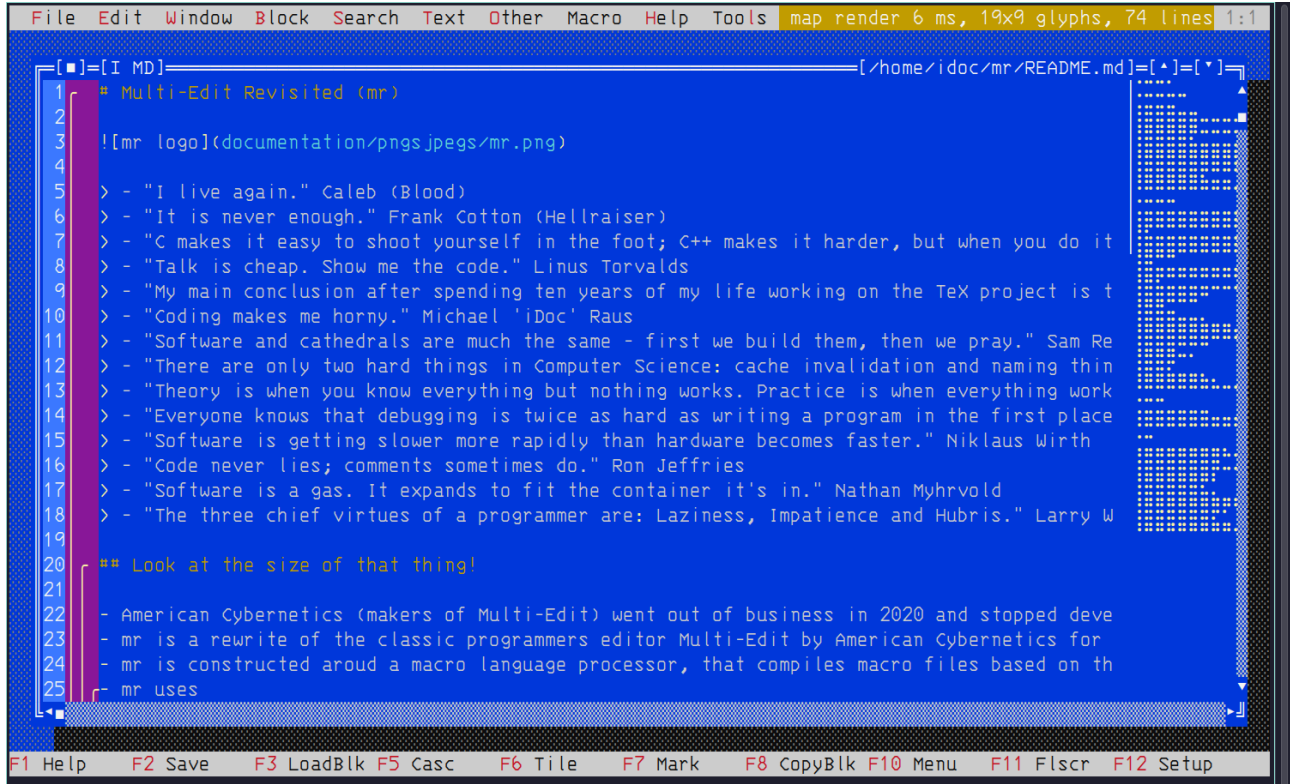


Figure 1.1: A normal editor window showing syntax-aware text, line numbers, folding, minimap, window markers, message area, and context-sensitive function-key labels.

1.2 Menus and the status line

The menu line is the complete compiled command map. Press **F10** to activate it, move with the arrow keys, and press **Enter** to execute a selection. A highlighted letter is the menu mnemonic. The mouse can also select menus and buttons when terminal mouse reporting is available.

The status line at the bottom shows the current function-key assignments. Those labels are contextual rather than decorative. Startup, ordinary editing, compiler diagnostics, file comparison, Bento panes, and the macro debugger may assign different meanings to the same function key. When this manual and the live status line disagree about a function key, the live status line wins. On terminals that report modifier state, holding **Shift**, **Ctrl**, **Alt**, or **Super** changes the displayed function-key bank before a function key is pressed. Empty positions mean that the current context has no assignment for that modifier combination. Label changes use the same short transition language as the message line.

Note. Most commands are reachable through both a menu and a keymap action. A key shown in a screenshot belongs to the captured profile and context; it is not a promise that every keymap uses the same sequence.

1.3 Help and explanatory messages

F1 opens context-sensitive Help for the active dialog or menu. **Shift+F1** opens the detailed Help index. Inside Help, **Tab** and **Shift+Tab** select references, **Enter** follows one, and **Backspace** or **Alt+F1** returns to the previous topic.

MR uses the menu line for transient information as well. Success, information, warning, and error messages have separate priorities and colors. Long-running work can post progress without replacing a more important error. The log window retains diagnostic detail that is too long-lived for the message line. Use **File > Open live log**, **File > Open journal**, or the configured log viewer when a short notice is insufficient.

1.4 Progress and performance observation

Some foreground batch operations enter Static Mode. The message lane then becomes a right-to-left progress bar with a centered **completed/total** count, ordinary transient messages are suppressed, and the function-key line changes to **Esc Abort**. **F1** through **F12** are disabled immediately outside Fullscreen; **Esc** belongs to the active operation. When the operation finishes or acknowledges cancellation, the previous labels and normal message animation return.

Help > Performance panel toggles a global diagnostic panel above the desktop. Its activity meter distinguishes active, queued, and result-ready work; the text rows show worker, lane, owner, timing, lifecycle, and selected runtime I/O counters. Use it to explain latency or verify that work returns to idle. It is an observation surface, not a task manager. **Esc** closes the panel.

1.5 Editor modes

Insert and overwrite determine how ordinary typing affects existing text. Read-only state prevents text mutation but still permits navigation, search, copying, and inspection. A binary file may open as a Bento hex editor instead of being decoded as text. File comparison and compiler output are also specialized workspaces rather than ordinary editable buffers.

The editor is UTF-8 capable. Display columns and byte positions are therefore not interchangeable concepts. Commands operate on the logical document and the active editor projection; users should avoid treating a displayed glyph as necessarily one byte or one terminal cell.

1.6 Fullscreen presentation

F11 gives the selected editor the complete desktop and hides its frame, the menu line, the function-key line, and the Performance panel. A brief hint shows the exit keys. **F10** exposes the menu temporarily without ending Fullscreen; **F11** or **Esc** restores the normal layout and the window's previous bounds. Fullscreen changes presentation, not document or workspace ownership. It also suspends Static Mode as a user-interface restriction: Static progress remains recorded for the operation but is not drawn, function keys remain usable, and **Esc** leaves Fullscreen rather than aborting the batch. If the batch is still running afterwards, its normal Static Mode display resumes.

1.7 A productive first session

A useful first exploration is deliberately small:

1. Open a source file with `File > Open`.
2. Move with the cursor and page keys and observe the line/column marker.
3. Press `F10` and inspect the top-level menus.
4. Open `Help > Keys` and compare it with the function-key line.
5. Mark several lines, copy them, and undo the copy destination.
6. Open `Window > List` to see the managed-window model.
7. Inspect `Other > Installation and setup` without changing a draft.

This route establishes the three recurring ideas of the manual: commands are contextual, visible UI state is authoritative, and setup dialogs stage changes before they become runtime settings.

Chapter 2

Cursor Control and Navigation

2.1 Character, line, and page motion

The arrow keys move by one logical editing position. `PageUp` and `PageDown` move by approximately one viewport. `Home` and `End` move to meaningful boundaries according to the active keymap, while dedicated top-of-file and bottom-of-file actions are available in the action catalog. Word-left and word-right use the configured word delimiters rather than an ASCII-only assumption.

The editor permits horizontal navigation beyond the currently visible columns. A horizontal scrollbar indicates the projection offset. Very long lines are not reformatted merely because they do not fit on screen.

Note. Keymap profiles may emulate WordStar, Emacs, Nano, or a personal scheme. Consequently this manual names navigation actions more often than fixed sequences. Use `Help > Keys` and the status line for the current binding.

2.2 Going directly to a line

`Search > Goto line number` opens a one-based line-number dialog. The accepted range is based on the document's known line state. On very large files the line index may still be completing in the background; MR reports estimated state until the exact result is available rather than freezing the editor.

The minimap is another direct navigation surface. Its viewport marker shows which fraction of the document is visible, and mouse selection can move the editor to a distant region. Search hits, changed regions, or diagnostics may add their own markers to the minimap.

2.3 Position markers

`Search > Push position` stores the current document location on the marker stack. `Search > Get position` returns to the most recent valid entry. This is useful before following a declaration, compiler diagnostic, or search result. Invalid locations are skipped when their windows or documents no longer exist.

Random-access marks provide named or indexed destinations where the active keymap exposes them. They differ from the push/pop stack: retrieving one does not mean “return to the immediately preceding place.” Use the stack for temporary excursions and random-access marks for landmarks that should remain available during a longer editing operation.

2.4 Matching and structural motion

`Other > Match parenthesis` moves between matching brackets or parentheses. Language-aware folding and outline commands provide larger-scale structural navigation. Their results depend on the detected language and the current syntax state; plain text remains navigable even when no structural producer is available.

Fold markers represent collapsible ranges without deleting text. Opening or closing a fold changes only the projection. Search, save, block operations, and MRMAC continue to address the underlying document.

2.5 Mouse navigation

The mouse can place the cursor, select menu entries, drag scrollbars, and activate dialog controls. Exact behavior depends on the terminal emulator and whether it forwards mouse events. Keyboard navigation remains complete and is the reliable fallback over remote or restricted terminal connections.

Chapter 3

Installation and Setup

3.1 One application, staged configuration

MR has no separate setup executable. Other > Installation and setup opens the configuration areas for colors, keymaps, filename extensions, compiler profiles, paths, backups, interface behavior, search defaults, and live logs. Each dialog edits a working draft. Accepting the dialog applies validated settings; cancelling it abandons the draft.

Persisted settings are represented in `settings.mrmac`. Users should change them through the dialogs or documented MRMAC settings operations. Directly editing serialized lines is possible for recovery and review, but it bypasses field validation and is not the normal setup path.

3.2 Keymap profiles

Installation and setup > Key mapping manages profiles and their bindings. A binding has a context, a target type, a target action or macro, a key sequence, and a description. Context prevents an editing binding from silently taking over a dialog or menu. Multi-key sequences are supported where the resolver and profile define them.

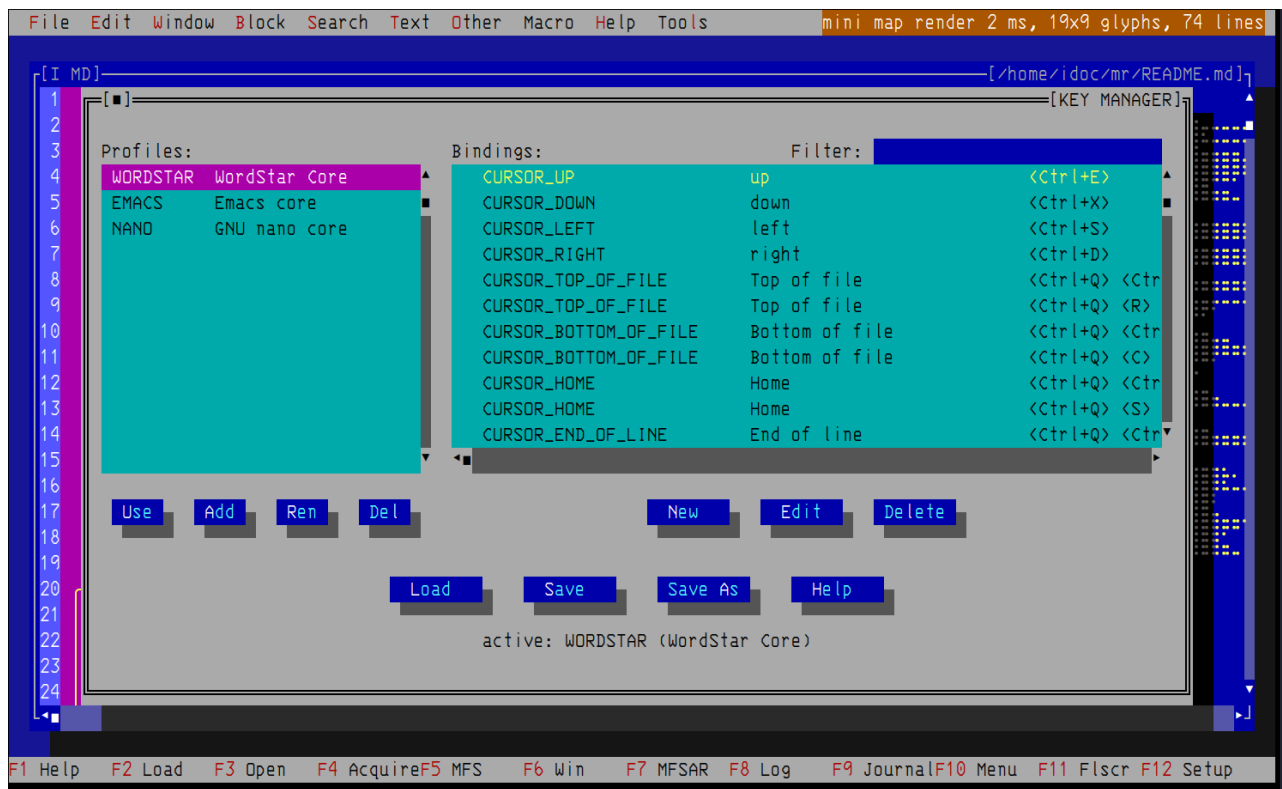


Figure 3.1: The Key Manager with WordStar, Emacs, and Nano profiles. The binding list belongs to the selected draft; Use makes a profile active.

Use activates the selected profile. Add, Ren, and Del manage profile drafts; New, Edit, and Delete manage

bindings. Load, Save, and Save As exchange profile files. Capturing a key is safer than spelling a sequence by hand because Help keys and reserved interactions are filtered by the capture dialog.

Before changing data. Do not delete or rewrite the only bindings needed to reach setup until the new profile has been tested. Keep a loadable known-good keymap file when experimenting with stateful or multi-key emulations.

3.3 Colors and themes

Installation and setup > Color setup groups palette roles by domain. Foreground, background, monochrome attributes, and the preview describe one selected role. Theme files can be loaded into the working palette and saved for reuse.

Filename-extension profiles may select a theme automatically. This allows a different visual treatment for code, prose, logs, or comparison work without creating a second global color registry. A loaded theme remains part of the dialog draft until the surrounding setup flow accepts it.

3.4 Paths

The Paths dialog configures the settings macro, macro directory, Help file, temporary directory, shell executable, audio player, and log destination. History limits control the retained file, path, and workspace entries.

Use absolute paths for infrastructure whose meaning must not depend on the launch directory. The temporary directory is for disposable products; do not point it at a source directory. The settings macro path must include a filename, not merely a directory.

3.5 Backups and autosave

Backup policy selects no backup, a sibling backup file, or movement to a backup directory. Frequency may be first save only or every save. Autosave can be driven by keyboard inactivity and by an interval; zero disables the respective trigger.

Workspace autosave is a distinct concern. It preserves window topology and workspace membership, while document autosave protects changed text. Restoring one does not imply that every dirty document has already been written to its persistent file.

3.6 User-interface and live-log settings

User-interface settings control the window manager, message line, cyclic virtual desktops, diagnostic adoption, binary autodetection, cursor behavior, scrollbars, comparison gutters, indentation preview, and desktop count.

Live-log settings control search-hit notices, beep or audio behavior, viewer line numbers, timestamps, syntax highlighting, and scroll direction. Audio is optional; unavailable controls are disabled instead of being silently accepted.

Chapter 4

Command-Line Use

4.1 Starting with files

Pass one or more file paths after the executable name:

```
./mr README.md Makefile src/main.cpp
```

Each readable file becomes an editor window. The last loaded window is active. Relative paths are interpreted from the launch directory and normalized before loading.

Shell glob patterns are accepted:

```
./mr '*.md' 'src/*.cpp'
```

Quote a pattern when MR, rather than the invoking shell, should expand it. Flat glob loading does not recurse into descendants.

4.2 Recursive loading

`-load-recursive` and its short form `-lr` enable recursive startup loading. A directory argument loads readable files below that directory. A wildcard argument recursively matches below the non-wildcard root.

```
./mr --load-recursive documentation
./mr -lr 'src/*.cpp'
```

Recursive loading can create many windows. Prefer a narrow filespec or a workspace when the repository contains build trees, vendored dependencies, or generated output that should not become part of the session.

4.3 Help

`-help` and `-h` print command-line help and exit:

```
./mr --help
```

The command-line help is intentionally narrower than the in-application Help. It describes startup parsing, while F1 explains the running editor.

4.4 Startup macro automation

Advanced workflows may run an MRMAC file after settings, AUTOEXEC macros, the workspace, and startup files have been loaded:

```
./mr README.md --run-macro=macros/session-start.mrmac
```

The two-part form is also accepted:

```
./mr --run-macro macros/session-start.mrmac
```

Multiple options run in argument order. The option `-exit-after-run-macro` requests termination after at least one startup macro has been processed.

This facility is useful for deterministic demonstrations, batch-like editor automation, and test sessions. It still runs through the normal MRMAC loader and UI-thread execution route; it is not a second command language.

4.5 Binary files at startup

When binary autodetection is enabled, boundary samples are checked before a startup file is assigned an ordinary editor. A detected binary is opened in the Bento hex editor. This prevents NUL-rich data from being rendered as meaningless text.

Note. Autodetection is a policy setting, not a claim that every non-text format is perfectly classified. The `Text > Hex editor` command remains available for an explicitly chosen text-to-hex transition.

Chapter 5

Loading and Saving Files

5.1 Open, Load, and Get Last

File > Open creates a new editor window for the chosen file. **File > Load** replaces the contents of the current suitable editor window. Use **Open** when both the current and new document should remain available; use **Load** when the present window is intentionally being reused.

File > Get last lists recently used files, folders, and workspaces. Empty groups are skipped when initial focus is chosen. Selecting a row performs the operation appropriate to its type rather than treating every history entry as a file.

5.2 Acquire, live logs, and the journal

File > Acquire accepts a directory specification or an executable command. Results are presented as selectable paths, and **Load all** opens every eligible entry. This is useful when the file set is discovered by a tool rather than known in advance. Output is adopted incrementally, duplicate paths are removed, and the list shows ordinals and a total. Closing the dialog while its command is running stops the process group and discards output that has not been adopted.

Load all processes one file boundary at a time and enters Static Mode. The message lane reports the number processed; **Esc** stops the remaining queue without closing files already loaded when Fullscreen is not active. In Fullscreen, leave the presentation first and press **Esc** again to cancel the remaining batch. The final notice distinguishes completion from cancellation and reports how many files were loaded.

File > Open live log follows a text file as it grows. **File > Open journal** reads `systemd` journal output, optionally restricted by an application tag. These are communication or log roles with lifecycle rules different from an ordinary source buffer.

5.3 Text and binary loading

Ordinary text files use the piece-table-backed editor. Binary files can use the Bento hex editor, whose synchronized panes show hexadecimal, decimal, binary, octal, strings, and typed inspector values.

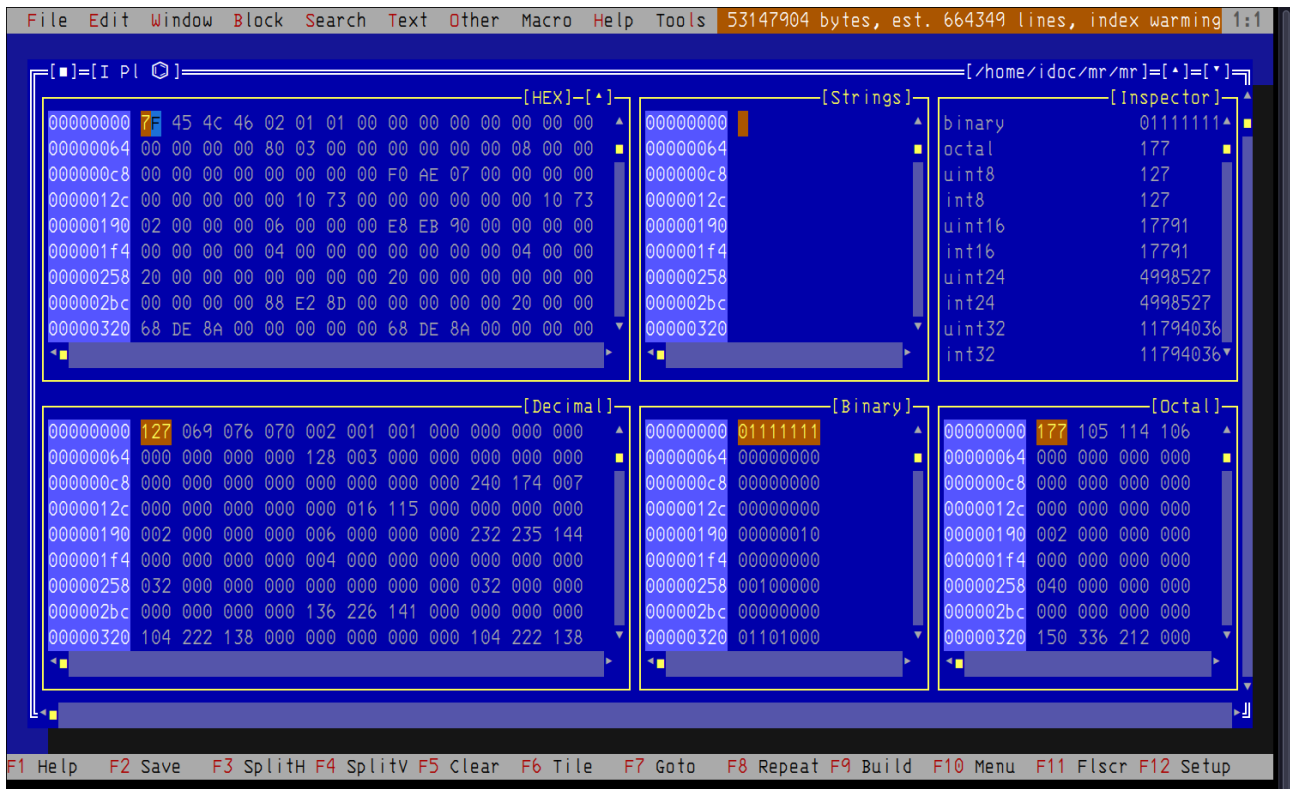


Figure 5.1: A binary executable opened from the command line. Hexadecimal, strings, inspector, decimal, binary, and octal panes share one position.

The hex editor is not a cosmetic mode applied to decoded text. It is a specialized workspace with panes that project the same binary source. Navigation in one pane updates the others. Use the inspector to interpret the selected bytes as signed and unsigned integer widths and related data forms.

5.4 Save, Save As, Save All, and Revert

File > Save writes the current document to its persistent path. File > Save As chooses another path and updates the window's persistent identity. File > Save all collects dirty windows and stops if a save is cancelled or fails.

File > Revert reloads the saved file and discards unsaved document changes after confirmation. It requires a persistent file. A read-only window cannot be saved as ordinary editable content, although supported log roles may offer their own snapshot save path.

Before changing data. Revert, closing a dirty window, loading a workspace over active state, and exiting are dirty-gated operations. Read the confirmation detail: Save, Discard, and Cancel are intentionally different lifecycle decisions.

5.5 File information

File > Info reports document identity, persistent path, size, line and cursor state, encoding-related editor state, window role, and runtime metadata. Use it when a file behaves differently than expected: the dialog is more reliable than inferring state from the filename alone.

5.6 Workspaces

A workspace records managed windows, paths, virtual desktops, geometry, visibility, main-file status, and supported Bento configuration. File > Open workspace loads such a description. The Window List offers workspace save/load controls and autosave/autoload toggles.

Workspace state does not replace file persistence. Saving a workspace records where and how documents are arranged; saving a document records its text or binary changes. A robust session workflow does both.

Chapter 6

Inserting Text

6.1 Insert and overwrite

In insert mode, typed text is added at the cursor and existing content moves logically to the right. In overwrite mode, eligible existing characters are replaced. The frame and menu state indicate the current mode. **Edit > Insert** toggles it.

New text enters the piece table’s add buffer. This implementation detail is visible to the user only through its consequence: inserting into a very large file does not require moving all later bytes in memory.

6.2 Lines and line endings

Enter inserts a line boundary using the active editor behavior. Filename-extension settings can require a final line ending, preserve or normalize relevant conventions, and control the EOF marker. Avoid assuming that a visually blank final row is the same as a stored final newline.

The CR/LF at EOF and related profile fields describe save policy, not an extra character painted into the text. **File > Info** helps distinguish document content from its projection.

6.3 Clipboard input

Edit > Paste inserts the current editor clipboard at the cursor. **Edit > Append** and **Edit > Cut and append** build a larger clipboard payload from several blocks. Terminal or operating-system clipboard exchange is available through the corresponding editor actions and MRMAC primitives.

Pasting into a column or line-oriented workflow should be preceded by a check of the current block state and insert mode. A persistent block can remain active after navigation; the highlighted projection is the warning that the next block-aware command may not act only at the cursor.

6.4 Unicode input

MR stores and displays UTF-8 text. A character may occupy several bytes, and a glyph may occupy zero, one, or more terminal cells depending on Unicode and terminal behavior. Cursor motion, column blocks, and formatting therefore use editor and display-column rules rather than byte offsets.

The MRMAC ASCII and Emoji tools can copy a selected character to the clipboard. They are convenient for characters that are awkward to enter with the terminal’s compose or input method.

6.5 Snippets and macro-generated text

The Snippet SideKick is a controlled editing surface for macro-generated expansion. **Tab** and **Shift+Tab** move among placeholders; **Ctrl+Enter** or **Alt+Enter** commits, while **Esc** cancels. The parent document changes only at commit.

This separation matters: a macro can prepare a complex expansion without mutating the document behind every intermediate keystroke.

Chapter 7

Deleting Text

7.1 Character deletion

Backspace removes the logical character before the cursor; Delete removes the eligible character at the cursor. At line boundaries the operation may join lines. UTF-8 sequences are treated as editor characters rather than a request to delete one arbitrary byte.

Read-only windows reject mutation. If a delete command appears inert, inspect the read-only marker, current window role, active block, and command state before assuming that input was lost.

7.2 Selections and blocks

Block > Delete removes the marked block. Edit > Cut also removes the selection but retains it in the clipboard. Choose Delete when no paste is intended and Cut when the text is being transferred.

Line, column, and stream blocks have different shapes. Deleting a column block removes a rectangular range on each participating line; deleting a stream block removes one continuous logical range; deleting a line block removes complete lines.

7.3 Deleting across windows

Block > Window move transfers the block to another editor window and removes it from the source after a successful destination operation. Window copy leaves the source in place. Use the explicit inter-window commands instead of assuming that an active clipboard implicitly names a destination.

7.4 Recovering a deletion

Deletion is recorded in the document's undo history. Use Undo immediately when the wrong range was removed, or traverse the history deliberately as described in Chapter 10. Saving does not erase the in-memory history by definition, but closing or replacing the document ends the current editor lifecycle.

Note. A backup file and Undo solve different problems. Undo reverses an in-session edit; a backup protects a previous persistent version. Configure both according to the cost of losing work.

Chapter 8

Tabs and Virtual Space

8.1 Tab width and expansion policy

The active filename-extension profile supplies the tab size and expansion policy. **Use tabs** inserts or preserves tab characters according to the operation; **Spaces** expands indentation into spaces. Tab size determines display stops and indentation width but does not change the byte value of an existing tab.

Mixed indentation is particularly sensitive to this distinction. Two lines can align visually under one tab width and diverge under another. Use language-appropriate profiles and code-formatting tools before performing large mechanical indentation changes.

8.2 Typing beyond end of line

Virtual space permits the cursor to occupy a display column beyond the stored end of line. When text is inserted there, the gap is materialized using the active tab and space policy. Merely moving through virtual space does not add bytes to the document.

This behavior is useful for column work and diagrams, but it can create unexpected whitespace when a cursor is placed far to the right accidentally. The **Truncate whitespace** profile option controls save-time treatment of unwanted trailing space.

8.3 Tab-aware blocks

Column blocks use display columns. A tab can span several cells, so a rectangular selection may intersect displayed tab space rather than a one-byte-per-column grid. Inspect the result after column moves, sorting, or indentation in tab-heavy files.

8.4 Indent and Undent

Block > Indent and **Block > Undent** apply the configured indentation to selected lines. Smart indentation may also use the language and nearby syntax. The operation is block-based and undoable; it is not a permanent rewrite of the profile.

8.5 Format line and margins

The format line provides a compact visual representation of tab stops and margins. It is covered in detail in Chapter 13. Tab setup, left and right margins, paragraph formatting, and centering should be treated as one formatting policy rather than unrelated numbers.

Chapter 9

Block Operations

9.1 Three block shapes

MR supports line, column, and stream blocks. A line block contains complete logical lines. A column block is rectangular in display coordinates. A stream block is one continuous range from its start position to its end position. The distinction determines what copy, move, delete, indentation, case conversion, and save-block operations receive.

Block > Mark lines, Mark columns, and Mark stream start the respective mode. End marking commits the current range. Turn marking off removes it. Hide/show block mark changes only whether the selection is projected; the block still exists when hidden.

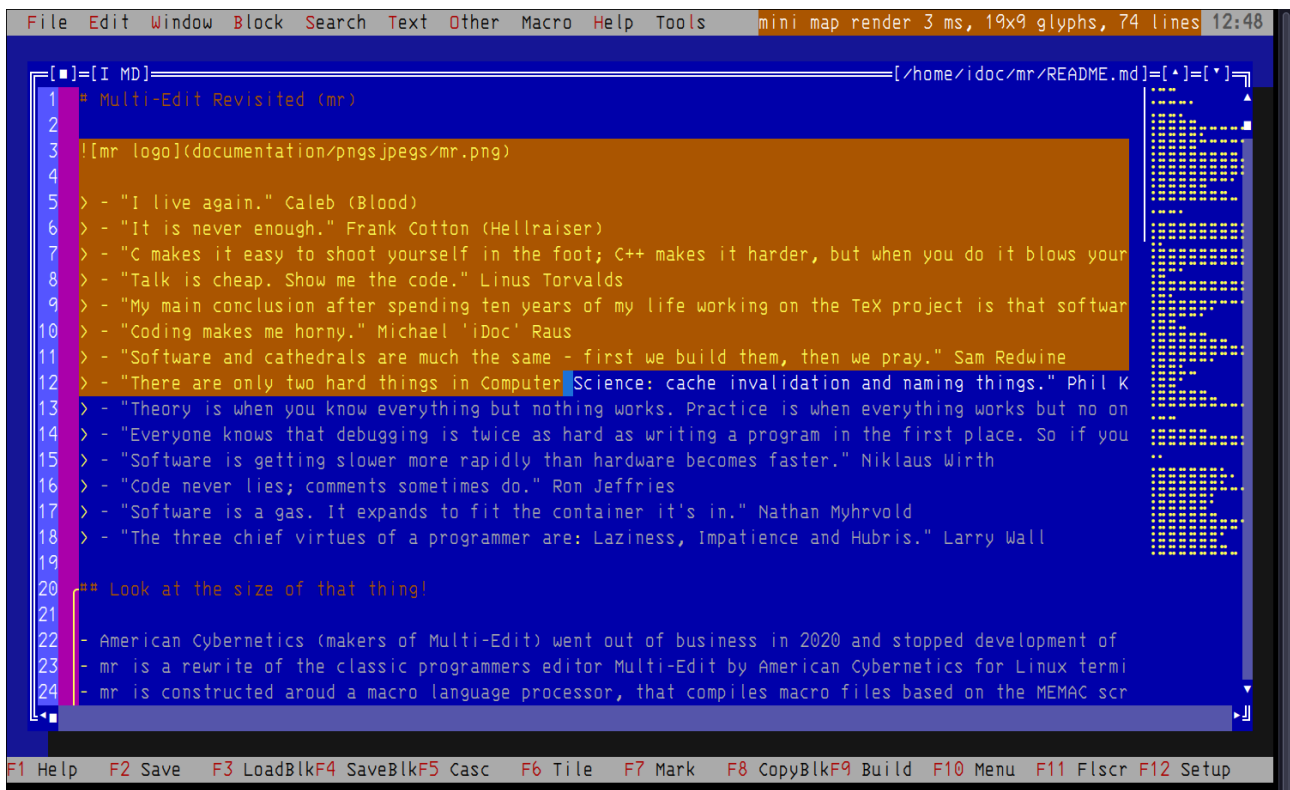


Figure 9.1: A stream block across several Markdown lines. The selected range ends at the cursor; the minimap and status information remain live.

9.2 Marking state and cursor motion

An actively extending block and a committed block are different states. Cursor motion extends an active selection. After commit, ordinary cursor movement may preserve or clear it according to the persistent-block setting. Block > Persistent blocks reports and changes that runtime policy.

The current block is a first-class editor state. A command may act on it even when the cursor has moved away or the visible highlight has been hidden. Before a destructive block command, verify the marked shape and endpoints.

Before changing data. Hidden selection is not absent selection. If a copy, delete, format, or case conversion affects more text than expected, turn block visibility on and inspect the persistent-block state.

9.3 Copy, move, and delete

Block > Copy inserts a copy at the command's destination while retaining the source. Move transfers the range. Delete removes it. Equivalent Edit-menu commands use the clipboard when appropriate.

Every successful mutation enters undo history as an editor operation. A large block does not need to be flattened into a second complete document merely to be moved; the piece-table model can express changes as new ranges and references.

9.4 Inter-window transfer

Block > Window copy and Window move explicitly transfer between editor windows. The target must support the requested mutation and must remain valid throughout the operation. A read-only or specialized target may reject the transfer.

Use ordinary clipboard copy when the destination is outside MR or when the transfer should survive closing the source. Use the window commands when the relationship between two managed editor windows is the important part.

9.5 Indent, Undent, and transformations

Block > Indent and Undent operate on selected lines using the active indentation profile. Text > Upper case and Lower case transform the applicable marked text. Code formatting may consume the block or the whole file according to the formatter command and profile.

Column-block sorting is available through its command or keymap action. The sort key is the selected rectangular slice, while complete participating lines remain the records being reordered. Confirm tab and display-column interpretation before sorting mixed-width data.

9.6 Loading and saving blocks

Block > Load block inserts text read from a file at the block position. Save block writes only the marked range. These commands are useful for reusable fragments and for exchanging data without replacing or saving the whole document.

A saved block is plain selected content, not a workspace or editor-state serialization. Its block shape, cursor location, theme, and profile are not encoded in the block file.

Chapter 10

Undo and Redo

10.1 Document-local history

Undo and Redo traverse the active editor's change history. Each document owns its history; changing windows changes which history the commands address. Read-only views can be navigated but do not gain a mutation history merely because Undo is invoked.

An undo entry represents an accepted editing operation, not every internal piece-table step. Text insertion, deletion, block operations, formatting, and supported macro edits therefore reverse at meaningful editor boundaries.

10.2 Traversing the history

Edit > Undo moves one step toward the earlier document state. Edit > Redo reapplies a step that was undone. A new mutation after Undo normally establishes a new forward history; do not assume that the abandoned redo branch remains selectable.

Watch the modified marker while traversing. Returning to content equal to the last saved snapshot can clear the dirty relationship even though the document has a non-empty history.

10.3 Undo and large files

The document model stores snapshots and piece relationships rather than copying the entire file for every operation. This makes practical undo depth possible even when the original mapping is large. It does not make history free: inserted text, metadata, and retained snapshots still consume memory.

Background syntax, folding, minimap, or line-index results are derived state. When Undo changes the document version, stale results are rejected and fresh work is scheduled. Undo therefore restores document semantics; derived projections catch up without owning the edit.

10.4 Undo versus persistent recovery

Undo exists only in the running document lifecycle. Backups, autosave, and version control protect different time horizons. Closing a document, restarting MR, or reverting from disk is not a substitute for explicitly saving an important state.

Note. For risky transformations, combine a saved file or version-control checkpoint with Undo. One protects the persistent baseline; the other supports rapid in-session experimentation.

10.5 Macros and undo

MRMAC editor procedures route mutations through the same editor model. A macro can group work according to its operations, and the user can undo accepted changes. A macro that performs external I/O, launches a process, or changes settings cannot make those external effects disappear through text Undo.

Chapter 11

Windows, Virtual Desktops, and Workspaces

11.1 Creating, closing, and selecting windows

Window > Open creates another editor window. Close closes the active managed window after dirty-state handling. Next and Previous traverse neighboring desktop windows. List opens the complete control dialog.

The Window List reports minimized or hidden state, virtual-desktop number, workspace-main status, file, and directory. It also offers Delete, Save, Save all, Save as, Revert, Hide, Get, and workspace commands. Use it when direct cycling is no longer an adequate model of the session.

11.2 Hide, minimize, and zoom

Hide removes a window from ordinary selection without closing it. Minimize reduces a supported window to its one-line desktop representation. Zoom toggles maximized geometry. These states are distinct and are shown differently in the Window List.

Restoring a minimized window returns it to stored normal geometry. Unhiding a window returns it to selection. Neither operation reloads the file or creates a new document.

11.3 Cascade, tile, and split

Window > Organize > Cascade overlaps eligible ordinary desktop windows. Tile partitions the usable desktop among them. Modal dialogs and Help do not participate.

Horizontal and vertical split create another viewport of the same supported document. Edits are shared because both panes refer to one document; cursor and scroll state belong to the individual viewport where supported.

11.4 Virtual desktops

MR supports one through nine virtual desktops. Viewport-left and viewport-right commands select the neighboring desktop. Move-window-left and move-window-right change the active window's membership and select the target desktop.

Cyclic navigation is configurable. With cycling disabled, commands stop at the first or last desktop. With cycling enabled, navigation wraps. The live command state disables operations that cannot currently produce a valid transition.

11.5 Links

Window > Link and Unlink control the MRMAC window link used by editor procedures. A link is not the same as a split viewport or file comparison. It gives macro operations an explicit related window without creating a second document ownership model.

11.6 Workspace lifecycle

Workspaces combine window identity, geometry, visibility, virtual desktops, main-file designation, and supported Bento state. The Window List can mark the main workspace file, save or load a workspace, and toggle autosave or autoload.

Get in the Window List moves a window from another virtual desktop into the current one. This is a topology change, not a file reload. Workspace load validates entries before adopting them; missing or invalid files are reported instead of being silently converted into unrelated empty windows.

Before changing data. Save dirty documents before replacing the active topology with a workspace. Workspace persistence records session organization; it is not a bulk Save command for document contents.

Chapter 12

File Formatting

12.1 Formatting scope

Formatting commands distinguish the active line, paragraph, marked block, and whole file. The presence of a block can change the intended scope. Before running a formatter, make the selection visible and confirm the active filename-extension profile.

Text > **Upper case** and **Lower case** transform selected text. **Center line** uses the configured left and right margins. **Re-format paragraph** wraps prose according to the formatting policy. **Re-format code** invokes the language formatter configured for the file.

12.2 Paragraph reformatting

Paragraph formatting is for textual layout, not language syntax. It joins and wraps the current paragraph within the active margins while respecting paragraph boundaries. It should not be used as a source-code pretty printer.

The left and right margins can be changed through their dialogs. Both values are one-based and must form a valid range. They are synchronized with the persisted format-line settings.

12.3 Code reformatting

Code reformatting delegates to the configured formatter pipeline for the detected language and file profile. A formatter may operate on the block or file and may be an external program. Its output is adopted through the editor mutation path so that successful changes remain undoable.

Do not run a formatter on unsaved generated output or an incorrectly detected language without first checking the profile. When the operation has external requirements, missing tools or invalid output are reported rather than treated as valid empty formatting.

12.4 Whitespace policy

Filename-extension profiles control tab expansion, tab size, truncation of trailing whitespace, final line behavior, margins, and formatting commands. Formatting should be viewed as an application of that policy, not a universal style hard-coded into the editor.

12.5 Exporting to PDF

File > **Export to PDF** replaces the historical idea of direct printer control with a portable output artifact. The dialog selects output path, header, footer, page separator, font family and size, text width, and page margins.

The export path is separate from the source path. Header and footer are repeated page text. The page separator is a literal document string that forces a page boundary when encountered. Margins are PDF point values; text width is a formatted column count.

Note. PDF export is a text export, not a screenshot of the terminal theme. Syntax colors, minimap, window frame, and dialog chrome are not the printable document.

12.6 File comparison

Text > File compare opens a comparison Bento from two suitable editor windows. Difference markers and minimaps present structure; next/previous change navigates it. Apply-original-to-compare and apply-compare-to-original transfer the selected difference in an explicit direction.

Read-only comparison policy can prevent accidental mutation of the compare side. The direction names are deliberately verbose: verify them before applying a large change.

Chapter 13

The Format Line and Margins

13.1 Purpose

The format line is a compact ruler stored in the filename-extension profile. It communicates tab stops and formatting margins to commands that need them. Unlike a drawn ruler in the document, it is editor configuration and does not become file content.

13.2 Reading the ruler

A typical format line uses visible markers separated by repeated filler characters. Exact glyphs are theme and configuration details; their positions are what matter. The current tab size, left margin, and right margin are also shown as explicit fields in the Filename Extensions dialog.

Use the ruler to detect a mismatch between expected and active formatting before indenting or wrapping a large range. A C++ profile and a prose profile will often have intentionally different policies.

13.3 Left and right margins

Text `> Set left margin` accepts a one-based value smaller than the right margin. `Set right margin` accepts a value greater than the left margin. Centering and paragraph wrapping use this interval.

Margins do not truncate long lines on load. They guide formatting operations. Text beyond the right margin remains part of the document unless a command explicitly wraps or changes it.

13.4 Tabs and the format line

Tab size and expansion policy interact with the ruler. A displayed stop is a column concept, while a literal tab is one character in the file. Changing the profile can change alignment without changing stored bytes.

For code, prefer the language and repository’s indentation policy. For tables or diagrams, verify alignment in the actual terminal width and with the intended tab settings.

13.5 Profile inheritance

The DEFAULT filename-extension profile supplies global fallback settings. A matching named profile overrides the relevant values. Consequently a format line that appears to “change by itself” usually indicates that the active file matched a different extension profile.

Chapter 14

Search and Replace

14.1 The search dialog

Search > Search for text opens the single-document search form. Search text may be a literal, a PCRE2 regular expression, or a word/phrase according to Type. Direction is forward or backward. Mode stops on the first occurrence, prompts for the next, or lists all occurrences.

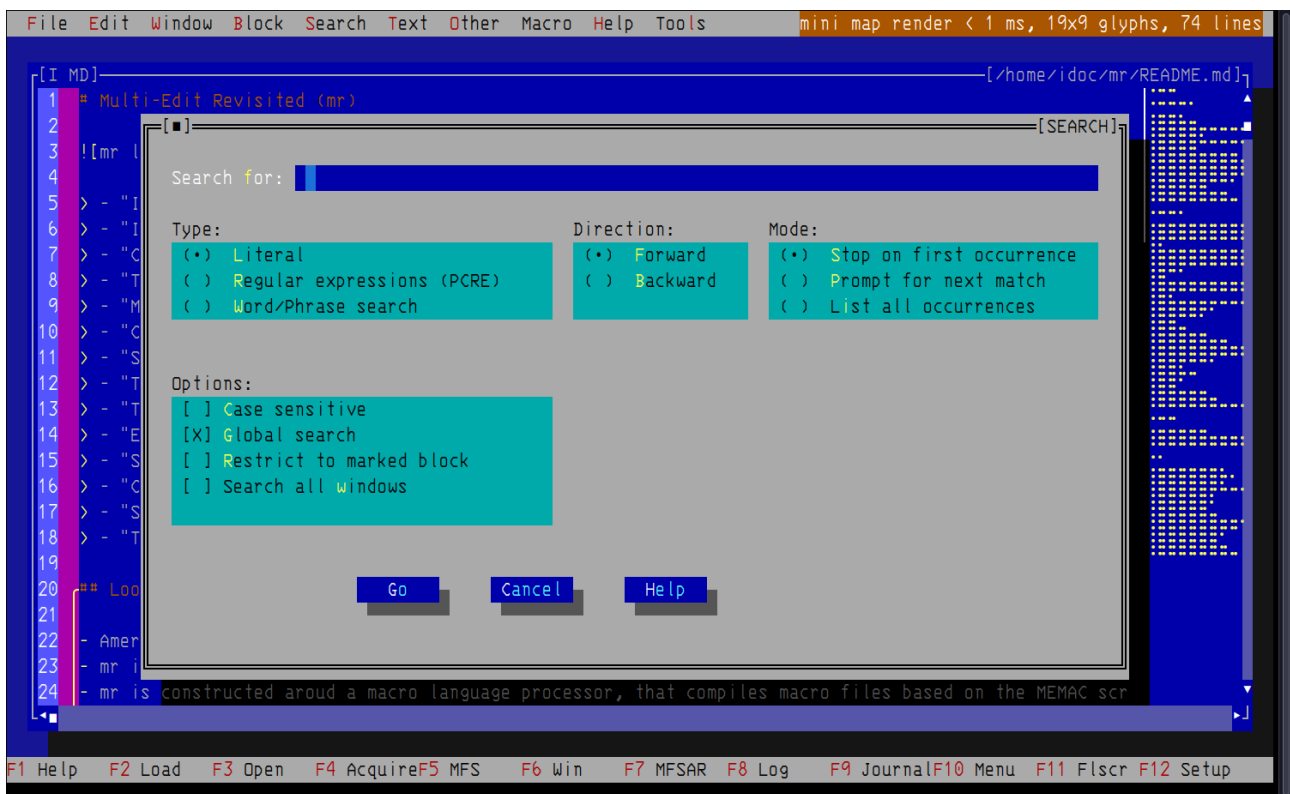


Figure 14.1: Single-document Search with literal, PCRE2, word/phrase, direction, result mode, and scope controls.

Case sensitive changes comparison. Global search controls boundary behavior. Restrict to marked block limits the range. Search all windows extends traversal to eligible editor windows. Cancelling preserves the previously accepted search specification.

14.2 Literal, word, and regular-expression searches

Literal search treats the entered text as text. Word/phrase mode applies word boundaries using editor rules. PCRE mode delegates pattern semantics to PCRE2. Do not escape a literal as though it were a regular expression unless PCRE mode is actually selected.

Regular expressions are powerful enough to match empty strings or very broad ranges. Test a pattern with stop-on-first or list mode before combining it with replace-all.

14.3 Replace

Search > Search and replace adds replacement text, policy, and cursor placement. Mode can replace the first match, prompt for each, or replace all. In prompt mode, Replace, Skip, Replace All, and Cancel make the decision for the current and remaining matches.

Regular-expression replacement syntax is active only with the corresponding search mode. Preview lines in the confirmation dialog are not an editable buffer.

Before changing data. Before Replace All, use list mode or a restricted marked block and inspect representative matches. For multi-file work, commit or back up the affected files first.

14.4 Repeating and continuing

Search > Repeat previous search reuses the last accepted specification without reopening the form. Continuation prompts distinguish Next, Stop, and Cancel. Search state is not rebuilt from partially edited fields in a cancelled dialog.

14.5 Results and navigation

List mode produces a Found List. Each row identifies document and location. Activating it loads or focuses the document and moves to the match. Push the current position first when following results should be a reversible excursion.

Search hits may be shown on the minimap and message line. The result location is resolved against the current document version; stale results are not allowed to overwrite or misdirect a newer document silently.

14.6 Multiple-file search

Search > Multiple file search selects filespec, root, recursion, search text, matching rules, editor-window inclusion, and optional workspace restriction. The request is immutable after submission and runs in the background. Collection initially reports an open-ended count; once the candidate set is known, the progress display reports files searched against the total. The Search button becomes Cancel while the operation is running. A cancelled or stale partial result is not opened as a valid Found List.

Search editor windows includes already loaded documents. Restrict to workspace limits candidates to the active workspace. Whole-word mode takes precedence over regular-expression mode when both controls would otherwise be selected. These options are independent: including loaded windows adds their current in-memory snapshots, whereas workspace restriction limits the candidate population to workspace members.

The result dialog can load the selected file or load every result file. Load keeps the result dialog available for further navigation; Load all opens each distinct result file without treating every match as another file load.

14.7 Multiple-file search and replace

The replace variant adds replacement text, review actions, and Keep all files open. Results are collected before UI adoption. Review permits Replace, Replace All, and Skip. Files need not all remain open unless that option is selected.

Replace All applies one replacement per dialog event and shows **completed/total** progress. Cancelling pauses mutation and offers Continue, Abort, or Revert. Continue resumes. Abort keeps the completed prefix as ordinary Undo records. Revert restores the pre-operation snapshot of every file touched by that Replace All run; it does not discard unrelated earlier edits.

Workspace restriction is especially useful in repositories with vendored or generated trees. A filespec alone describes names; workspace membership describes the intended project set.

Chapter 15

Keystroke Macros

15.1 Recording

Macro > Recording start/stop captures eligible key activity into a new macro source. The frame displays a recording marker. Stopping the recording offers to bind the result to a key.

Help-navigation keys and the recording toggle itself are treated specially so that a macro does not accidentally record the command that terminates its own capture. Modal interactions may be represented by resulting commands rather than raw terminal bytes.

15.2 Naming and binding the result

A recorded macro becomes a normal MRMAC definition that can be inspected and edited. The binding dialog captures a key or modified sequence; `Esc` leaves the macro unbound. Binding is separate from creation, so a useful macro need not consume a key.

Use the Key Manager to review conflicts. If more than one profile or context defines a sequence, the active resolver semantics determine which target is eligible. Descriptions should state what the macro does, not merely repeat its filename.

15.3 The Macro Library

Macro > Macro Library lists indexed files, definitions, bindings, and AUTOEXEC membership. Create, Delete, Copy, Edit, and Bind operate on the selected definition. AUTOEXEC Add and Remove control which macro files run during startup.

Playback runs the selected macro normally. Debug starts it in the MRMAC debugger, where breakpoints, watches, evaluation, stepping, and execution state are visible. Use Debug for authored logic; use ordinary Playback for a known routine.

15.4 What recording does not capture

Recording is not a video or terminal-event dump. It produces a macro that replays supported editor meaning. External program state, wall-clock timing, network responses, and files changed by another process are not automatically made deterministic.

When a recorded sequence becomes a maintained tool, edit its source to add validation and explicit error handling. Repeatedly recording a longer sequence is less reviewable than turning stable behavior into a named MRMAC routine.

15.5 Saving and reloading

Macro files live under the configured macro directory or another chosen path. The library indexes available definitions. A saved keymap binding and an AUTOEXEC entry refer to macro identity; moving or renaming sources can invalidate those references.

Chapter 16

Line Drawing

16.1 Enabling the mode

Text > Line drawing toggles line-drawing mode for the active editor. The menu reports ON or OFF. Double lines is available only while line drawing is enabled and selects the preferred double-line style.

Line drawing inserts Unicode box-drawing glyphs into the document. It is not a temporary screen overlay. Saving the file preserves those characters as UTF-8.

16.2 Drawing with cursor motion

With line drawing active, the supported modified cursor gesture draws between the old and new positions. Existing adjacent line-drawing cells are inspected so that corners, tees, crossings, and mixed single/double connections receive an appropriate glyph.

The editor can materialize rows and virtual columns when drawing beyond existing text. This is useful for diagrams but can add whitespace and new lines. Undo remains available for each accepted drawing operation.

16.3 Drawing with the mouse

A plain left-button drag draws a connected path. Direction changes are resolved into orthogonal segments. Fast collinear motion is coalesced so the document receives the final intended line instead of every intermediate mouse sample.

Terminal mouse support must be active. Over remote connections with unreliable mouse reporting, cursor-based drawing is more predictable.

16.4 Rectangles from column blocks

In line-drawing mode, an eligible column-block gesture can produce a rectangle whose corners are the block bounds. The operation materializes empty rows or columns as needed and consumes the drawing selection after the box is committed.

16.5 Single and double intersections

Double-line preference applies to the new connection. When a new segment meets existing line-drawing text, MR preserves supported neighboring connections and chooses a same-style or mixed glyph. Unsupported arbitrary characters are not guessed into a box-drawing topology.

Note. Line drawing works best in a monospaced terminal font with complete Unicode box-drawing coverage. Misaligned display can be a font or terminal-cell issue even when the saved UTF-8 characters are correct.

Chapter 17

Calculator

17.1 Opening the calculator

The bundled desktop-utilities macro adds Tools > Calculator. Its default binding is Alt+Shift+C. Because the tool is supplied by MRMAC, the menu item is available after the corresponding utility macro has been loaded and registered. An installation that deliberately omits that macro will not show the command.

The calculator is a modal work surface: the active document remains visible behind it, but editor input resumes only after the dialog is closed or its value is pasted. The upper display is the current input; the second display shows the memory register.

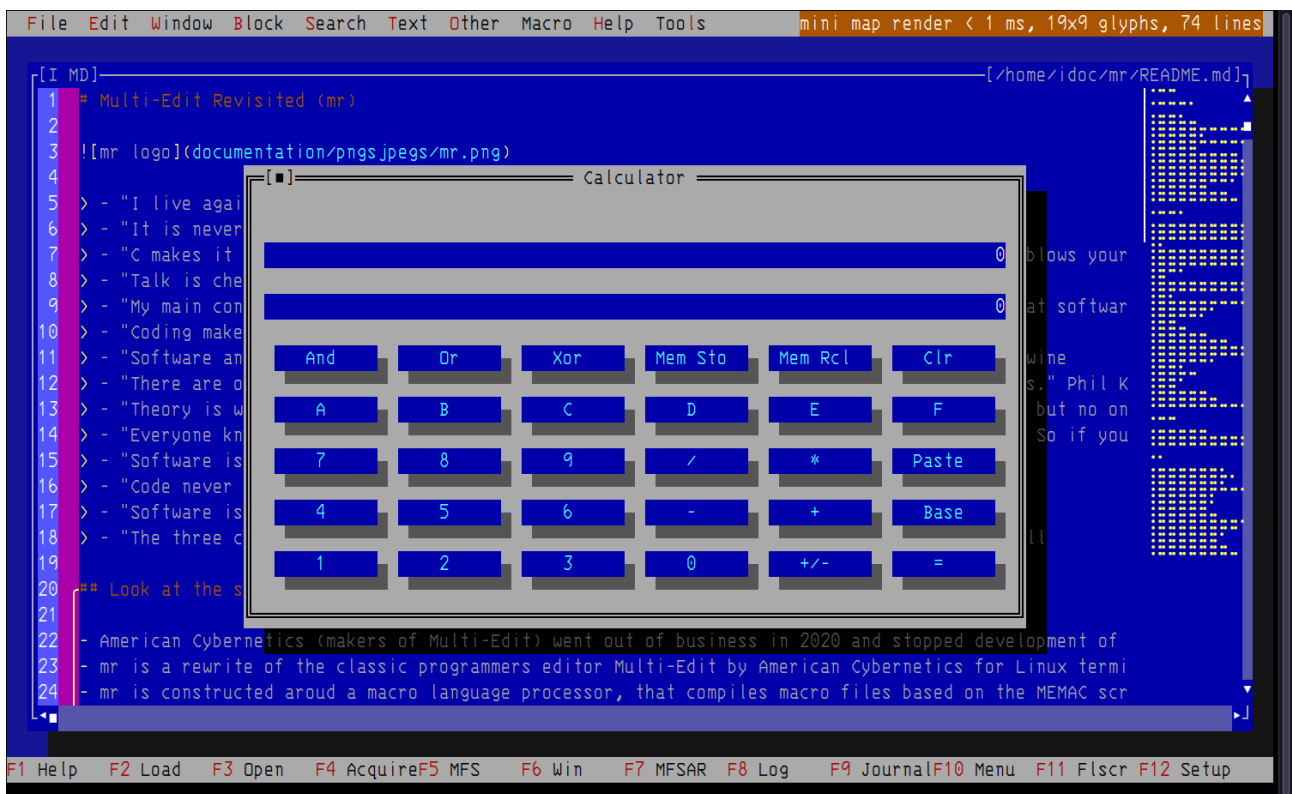


Figure 17.1: The bundled MRMAC calculator in a real MR PTY session.

17.2 Entering values

The digit buttons enter a value in the current base. Base cycles through decimal, hexadecimal, and binary representation while preserving the numeric value.

Hexadecimal output uses a leading dollar sign, for example \$2A. The buttons A through F are accepted only in base 16. In binary mode, only zero and one are valid. +/- changes the sign of the current input; Clr clears the input and clears memory when the displayed value is already zero.

If a stream or line block contains a valid decimal integer when the calculator opens, that number seeds the input. Whitespace is ignored for this purpose. Column blocks and non-numeric selections are left untouched and the calculator starts at zero.

17.3 Arithmetic and logical operations

The four arithmetic operators perform integer addition, subtraction, multiplication, and division. Division is integer division. And, Or, and Xor operate bitwise on integer operands.

The interaction follows a conventional accumulator:

1. enter the first operand;
2. choose an operator;
3. enter the second operand;
4. choose another operator to continue, or = to display the result.

The calculator rejects division by zero, invalid digits for the selected base, and malformed pasted input. A rejected operation leaves the dialog open so the value can be corrected.

17.4 Memory and pasting the result

Mem Sto copies the current numeric value to the memory register; **Mem Rcl** recalls it in the active base. The memory display is right-aligned beneath the input display.

Paste inserts the displayed representation at the current editor position and closes the calculator. This is a document edit and participates in normal Undo and Redo. In hexadecimal mode the inserted text includes the dollar prefix; change the base before pasting if the destination expects another notation.

Note. The calculator is an integer tool, not a floating-point expression evaluator. For repeatable calculations embedded in an editing workflow, an authored MRMAC routine is usually the clearer instrument.

Chapter 18

Character Tables

18.1 ASCII and Latin-1

Tools > ASCII table, normally bound to **Alt+Shift+A**, opens a modeless table. Each row contains the visible form, its decimal code, and the range name. Codes 0 through 127 are the ASCII range; codes 160 through 255 are presented as Latin-1. The unused C1 interval from 128 through 159 is not listed.

Control codes receive readable labels. For example, code 1 appears as `^A`, space appears as `SP`, delete as `^?`, and non-breaking space as `NBSP`.

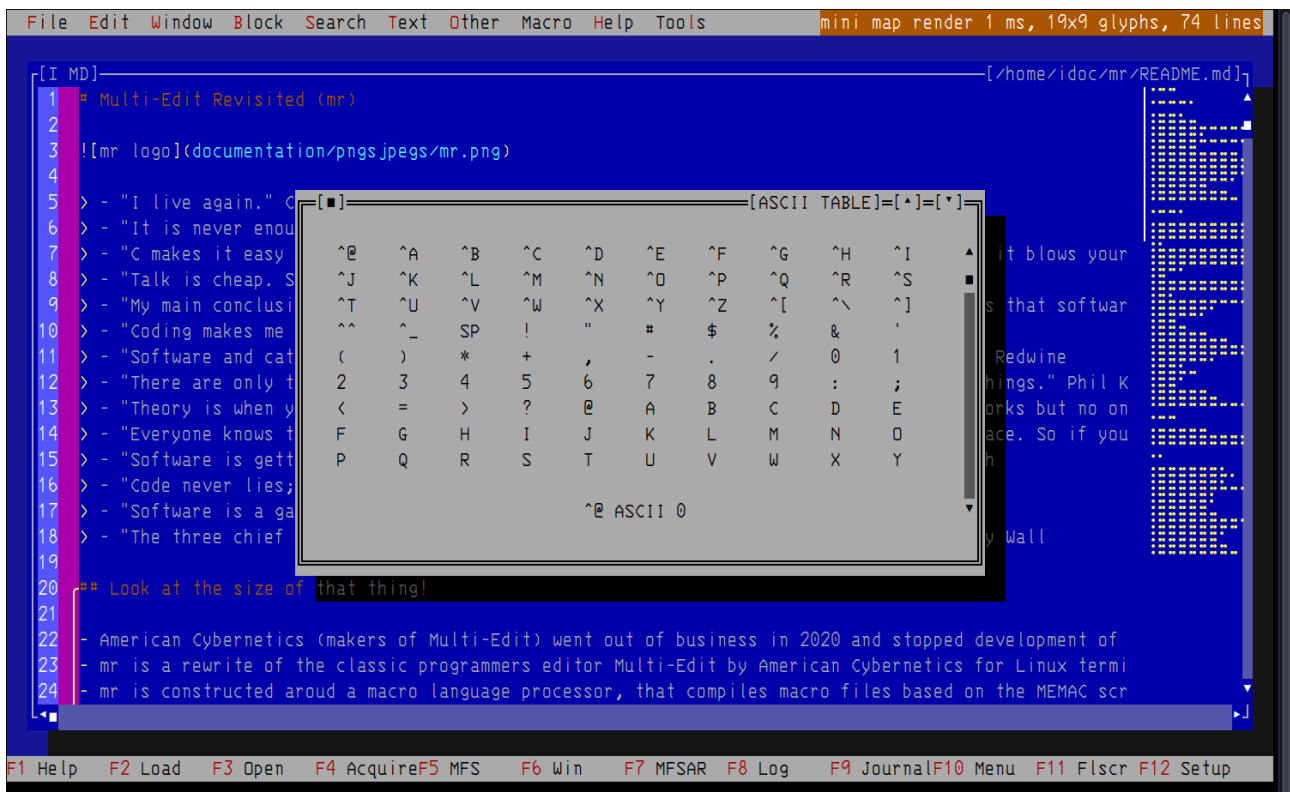


Figure 18.1: The modeless ASCII and Latin-1 table; the decimal code is shown beside each character.

Choosing an entry copies the corresponding character to the system clipboard and closes the table. Paste it with the active keymap's clipboard command. The selected row is remembered when the table is opened again during the same session.

18.2 Emoji and Unicode symbols

Tools > Emoji table, normally bound to **Alt+Shift+E**, works in the same way. It presents selected Unicode symbol and emoji ranges, including technical symbols, dingbats, regional indicators, pictographs, emoticons, transport symbols, and newer symbol blocks. The right-hand column gives the U+ code point.

Selecting a row copies the UTF-8 character to the clipboard. Whether the glyph is visible in the table and editor depends on the terminal font and its fallback fonts. A missing-glyph box is a rendering limitation; it does not mean that the copied UTF-8 sequence is invalid.

Note. The table inserts a character, not an ASCII escape notation. To write a literal numeric escape such as `\u263A`, enter that source-language syntax directly.

Chapter 19

Dense Display and Information Projection

19.1 Keeping document space visible

The historical idea of a condensed display is more useful in MR as a choice of information density. The editor can project line numbers, folding controls, difference markers, a minimap, scrollbars, a status line, and function-key assignments around the document. These elements do not become part of the file.

Use `Other > Installation and setup > User interface settings` to choose the global projection. Filename-extension profiles can then adapt relevant editor behavior to a file class. Keep enough context to navigate safely, but do not spend half a narrow terminal describing the other half.

19.2 Gutters

Gutter slots are ordered projections. Their setup codes are:

Code	Projection
L	line numbers
C	code-folding controls
D	difference markers in compare views
M	minimap

The left-to-right order in the setup dialog is significant. A blank slot removes that projection. Line numbers refer to document lines, while a folding control describes structural state derived for the current language.

19.3 The minimap

The minimap is a sub-character overview of the whole document. It provides orientation in files much taller than the viewport and can project notable regions such as diagnostics or changes. Selecting or dragging in it moves the editor viewport without changing the document.

The minimap is deliberately a summary, not a second editor. Use it to answer “where am I?” and use the primary viewport to inspect exact text. Syntax and document changes may be computed asynchronously; a newly loaded large file can therefore acquire its complete overview progressively.

19.4 Folding and zoom

Code folding hides the interior of a derived structural region while retaining its lead line. Fold and unfold commands affect only the view. Edits, saves, searches, and builds continue to operate on the underlying document.

`Window > Zoom` maximizes or restores the active desktop window. `Window > Minimize/restore` changes it to or from its one-line desktop representation. `Window > Hide` removes the window from normal selection without closing it. These are window-state operations; none discards unsaved text.

19.5 Terminal size and remote sessions

When the terminal becomes smaller, prioritize the editor viewport and retain only the projections needed for the task. Function-key labels and the message line are valuable when learning a keymap; experienced users may prefer more document columns. On a remote connection, disabling visually busy projections can also reduce terminal traffic without changing editor semantics.

Chapter 20

Filename-Extension Profiles

20.1 Why profiles exist

A filename suffix is more than a syntax-color hint. In MR, a filename-extension profile groups the behavior expected for a class of files: language, indentation, tabs, margins, display, block behavior, compiler profile, color theme, path, and optional lifecycle macros. This keeps a C++ source file, a Makefile, and a plain-text note from sharing inappropriate defaults merely because they occupy the same desktop.

Other > Installation and setup > Filename extensions opens the profile editor. The dialog works on a draft. Its Help describes the selected fields without applying the draft.

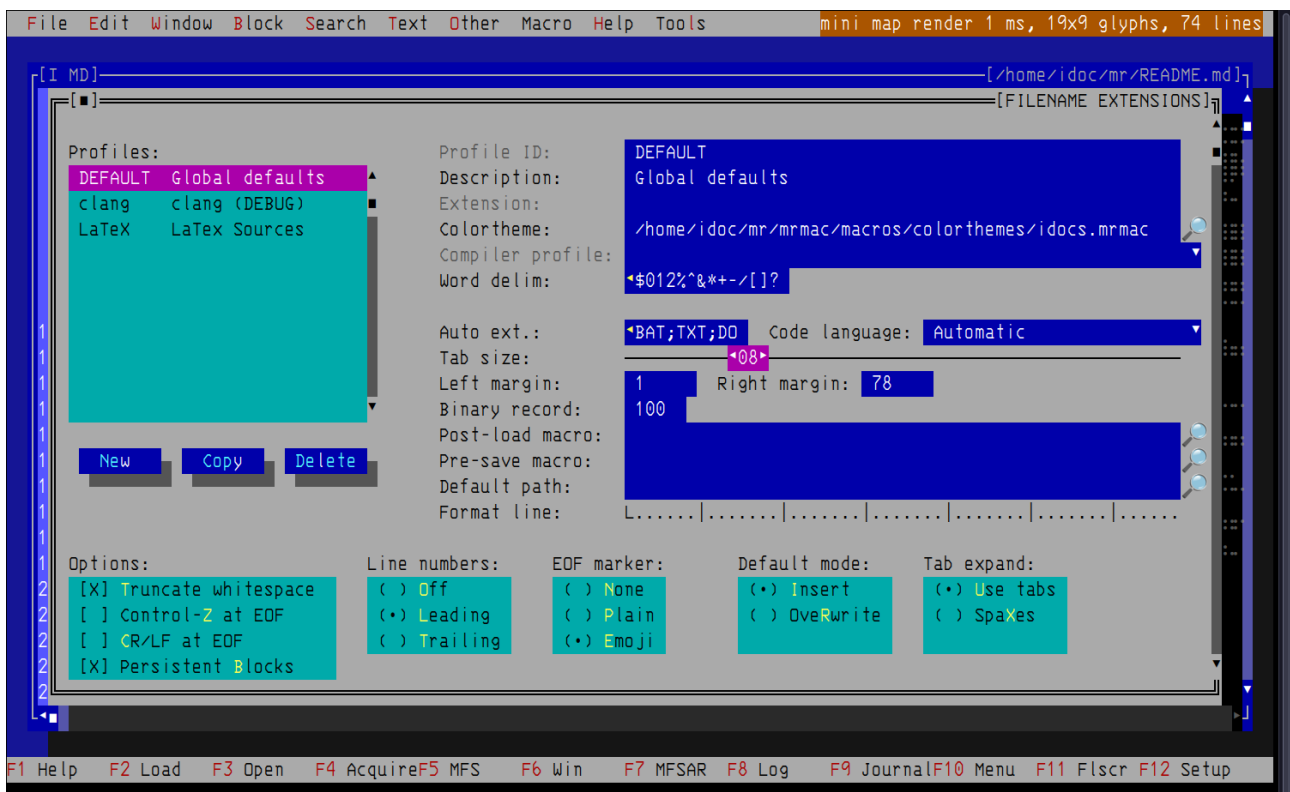


Figure 20.1: Filename-extension profiles combine matching, language, editor, and build choices.

20.2 Identity and matching

Profile ID is the stable settings key; Description is the human-readable name. Extension contains the exact, case-sensitive extension list used for matching.

The DEFAULT profile supplies global fallback settings. Its identity and extension selector cannot be edited. Specific profiles override the fallback when their selectors match. Avoid selectors that differ only by an easy-to-miss case unless the filesystem convention genuinely requires that distinction.

Before changing data. Changing a profile selector changes which future file loads receive that profile. It does not rewrite filenames, and it should not be used as a batch conversion mechanism.

20.3 Language and editor behavior

The profile controls language recognition and associated editor choices, including indentation, tab behavior, margins, format-line state, display, blocks, and editing modes. These choices establish a starting configuration; commands used in the active editor may still alter view or session state.

Binary detection is a separate concern. When Autodetect binary files is enabled in User Interface Settings, a binary file is directed to the Bento hex editor instead of being interpreted as ordinary text. That applies to files supplied on the command line as well as those opened later.

20.4 Build and lifecycle integration

Compiler profile associates matching files with a build description. Other > Build current file uses that relationship to choose a toolchain. A missing or incorrect association can therefore produce the wrong build command even when language highlighting looks correct.

Post-load macro names a MRMAC file to run after a matching file has loaded. Pre-save macro runs before it is saved. Typical uses include project-specific normalization or generated annotations, but these hooks are executable policy: keep them small, deterministic, and reviewable.

Default path supplies the initial directory for relevant file operations. Colortheme selects the palette associated with the profile.

20.5 Managing profile drafts

New creates a profile, Copy starts from the selected profile, and Delete removes a draft profile. Copying is the safest way to create a close variant because the full behavioral set remains visible for review. Accept the outer setup flow to make intended changes effective; Cancel leaves the runtime configuration unchanged.

Chapter 21

Indenting Styles

21.1 Automatic indentation

Smart indentation derives the leading whitespace for a new line from the active language and nearby structure. The filename-extension profile selects the relevant indentation behavior, while User Interface Settings offers an indent-style preview.

Automatic indentation is assistance, not a parser rewrite. It acts at the editing boundary and leaves already existing lines alone. This matters when opening generated code, patches, or files whose local style intentionally deviates from the project default.

21.2 Tabs, spaces, and visual columns

An indentation policy must distinguish three concepts:

- the number of visual columns in one indentation step;
- whether newly produced indentation uses tabs, spaces, or a defined combination;
- the display width assigned to an existing tab character.

These values often coincide in simple profiles but they are not equivalent. Changing tab display width can change alignment on screen without changing a single byte. Replacing tabs with spaces changes the document and should be a deliberate formatting operation.

21.3 Indenting and unindenting blocks

Block > Indent and Block > Undent adjust the lines intersecting the selected block by the configured indentation step. A stream selection therefore operates on all touched lines; a line selection makes the intended scope explicit.

Undent removes only eligible leading indentation. It does not delete arbitrary non-whitespace text to force a visual column. The operation is undoable as a document edit.

21.4 Language-sensitive structure

For brace-oriented languages, smart indentation observes opening and closing structure. For indentation-oriented languages, leading whitespace itself is semantically significant and deserves more caution. Macro sources, shell languages, markup, and plain text each have distinct structural cues.

When imported text has mixed indentation, first inspect tabs visibly and select the correct profile. Repeatedly pressing Indent to make the screen look right can preserve hidden inconsistencies. Use code formatting where a trusted formatter profile exists.

Note. If an indentation result is surprising, check the active filename-extension profile before changing global settings. A local profile mismatch is much more common than a universal indentation rule suddenly becoming philosophical.

Chapter 22

Language-Specific Features

22.1 Detection and syntax projection

The matched filename-extension profile establishes the language used by the editor. Language selection drives syntax highlighting, structural folding, smart indentation, outline production, and other derived views. Plain text remains a valid explicit fallback.

Syntax colors and fold markers are projections derived from a document snapshot. They are not stored in the file and are not part of Undo. On a large document, the visible area is prioritized while background work completes the wider derived state.

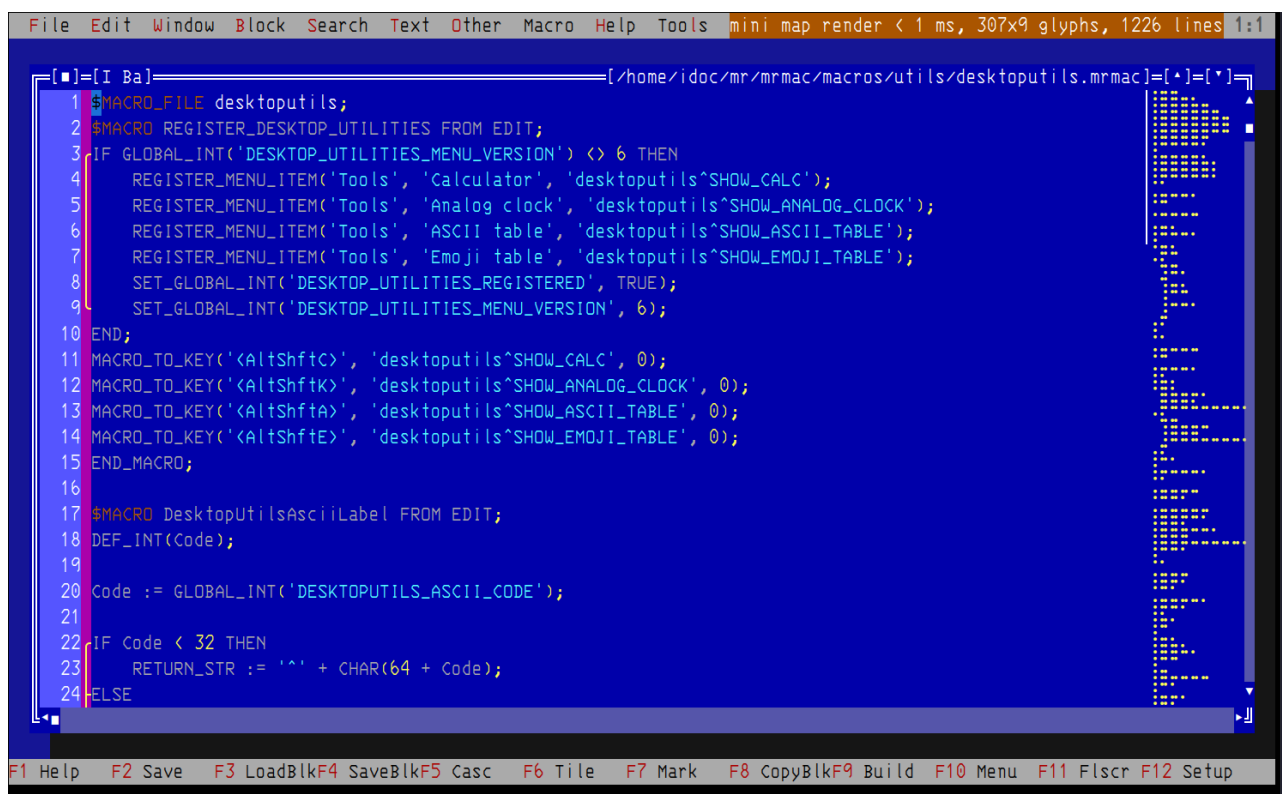


Figure 22.1: A MRMAC source file with language-aware highlighting, folding controls, and the minimap.

22.2 Code folding and outlines

Folding regions reflect structural constructs appropriate to the language: functions, types, sections, macro bodies, build targets, markup elements, or indentation scopes. Fold-level commands provide a coarse overview; individual gutter controls expose local structure.

An outline is a navigation model, not a second copy of the source. Its entries identify relevant document locations and may distinguish functions, methods, types, modules, sections, and other structural kinds. After

edits, allow the derived state to catch up before assuming that a temporarily absent entry has been deleted from the document.

22.3 Parentheses and structural navigation

Other > Match parenthesis moves between a supported matching pair near the cursor. It is preferable to a text search because it follows the language-aware structural pair rather than the next identical glyph.

Search markers, compiler diagnostics, fold outlines, and the marker stack provide complementary navigation. Use the narrowest semantic operation available: a diagnostic command for a compiler error, matching for a bracket, and search for textual occurrences.

22.4 Formatting source code

Text > Re-format code invokes the formatter configured for the active language or profile. This is distinct from paragraph reflow: a source formatter understands language syntax and can change whitespace throughout the applicable scope.

Before changing data. Review formatter output as a real document change. Save or commit a clean baseline before reformatting a large file, and use Undo immediately if the wrong profile or formatter was selected.

22.5 Compiler profiles and diagnostics

Other > Build current file chooses the compiler profile associated with the active file. A compiler profile describes the executable, toolchain, version and target constraints, build flags, include and library arguments, runtime paths, a pre-build shell command, build-succeeded and build-failed shell commands, and optional pre- and post-build MRMAC hooks. Separate success and failure audio paths can announce the result through the configured audio player.

Installation and setup > Compiler profiles exposes these fields and the **Automatic Setup** action. If the selected profile names an executable, Automatic Setup resolves and probes it, then fills toolchain, version, target, default flags, include paths, library paths, and runtime paths. With no executable selected, the action discovers supported compilers on **PATH** and inserts or selects matching profiles. The result remains an ordinary editable profile; automatic setup neither discovers a whole build system nor hides its decisions.

The pre-build MRMAC hook runs before the external build is submitted. The shell pre-build command can stop the build with a failing status; the matching success or failure command then follows the compiler result. Completion audio and the post-build MRMAC hook are adopted with the final result. Build context values identify the source, output, profile, toolchain, and exit status to hooks without requiring them to reconstruct the command line.

Other > Next compiler error and **Previous compiler error** navigate adopted diagnostics. User Interface Settings controls whether warnings and notes are tracked in addition to errors and how diagnostic messages are presented. The minimap can provide long-range orientation, but the diagnostic navigation command remains authoritative for the next adopted location.

22.6 Git status and file changes

For a named file in a Git work tree, MR probes the file's status in the background. The frame displays its Git changes marker when the file differs from the repository state or is untracked. Hovering explains the marker; clicking it is equivalent to **Other > Git changes**.

Git changes is file-scoped. If the editor contains unsaved changes, MR offers to save them first and stops if they are not saved. It then opens a communication window containing the repository and branch, short repository status, and the diff for the focused file. An untracked file is compared with an empty source. The command does not stage, commit, switch branches, or maintain a second repository model; use normal Git tools for those operations.

22.7 File comparison

Text > File compare opens a compare view. Difference gutters and the minimap summarize changes, while **Apply original to compare** and **Apply compare to original** transfer the selected difference in the named direction.

Read-only comparison can be enabled in User Interface Settings. Direction matters: the two apply commands are intentionally explicit because a compact “accept” command would conceal which document is being changed.

Chapter 23

The MRMAC Interface

23.1 What MRMAC contributes

MRMAC is the integrated macro language for editor automation and runtime UI extension. Its compiler runs in memory and supports both precompiled and on-demand macro files. A macro can edit text, inspect editor state, register menu items and keys, construct dialogs or modeless tools, start asynchronous work, and cooperate with the application's window system.

The calculator and character tables in Chapters 17 and 18 are concrete examples: they are application-quality utilities implemented as macros, not hard-coded simulations in this manual.

23.2 Macro files and definitions

A source begins with a macro-file identity and contains named macro definitions. The Macro Library indexes those definitions independently of their key bindings. One file can therefore provide several commands, shared helper routines, and a startup registration macro.

Use **Macro > Macro Library** to create, inspect, copy, delete, edit, bind, play, or debug indexed definitions. The editor provides syntax highlighting, folding, and outline support for `.mrmac` sources just as it does for other supported languages.

23.3 Execution contexts

A definition declares the context from which it may run. Editor procedures operate through the current macro/editor linkage; application procedures can address wider runtime facilities. A window may be explicitly linked or unlinked through the Window menu for procedures that need an editor target.

Macro execution is scheduled alongside interactive work. Long-running operations should use the language's cooperative facilities and explicit asynchronous interfaces instead of blocking the UI. Multiple jobs may be active, but shared document and UI effects still require deterministic ordering from the macro author's point of view.

23.4 Menus, dialogs, and modeless tools

Runtime menu registration adds commands owned by the defining macro. A conventional macro dialog is assembled, executed, and read as one interaction. A modeless macro tool stays on the desktop and can receive subsequent events while other windows remain usable.

Labels in a macro-created UI describe the macro's contract; MR does not invent semantics for them. Use the context Help available to the runtime UI where supplied, and consult the owning macro when a command is project specific.

23.5 Keys, recording, and AUTOEXEC

`MACRO_TO_KEY` and the Key Manager associate a macro definition with a key sequence and dispatch context. The Macro Library exposes the same relationship at a user level. Recorded keystroke macros become ordinary source definitions and can be refined after recording.

AUTOEXEC membership identifies macro files that run during startup. It is appropriate for menu registration, stable bindings, and deliberate session initialization. It is not a substitute for settings persistence, and startup macros should not silently rewrite unrelated configuration.

23.6 Running a macro from the command line

The command-line option `-run-macro=FILE` loads and executes a macro after application startup. It is useful for repeatable demonstrations, project entry actions, and controlled automation. The ordinary application environment still applies: the named file must be readable, any required editor target must exist, and errors are reported through the normal macro runtime.

This manual's screenshots use that mechanism to open deterministic UI states inside real PTY sessions. The resulting dialogs were not painted into the images afterward.

23.7 Debugging and reference material

Choose Debug in the Macro Library to start a definition under the integrated debugger. Breakpoints, stepping, watches, expression evaluation, and execution state are intended for authored logic whose behavior cannot be understood from one playback.

The MRMAC Reference Manual defines lexical rules, types, control flow, procedures, editor primitives, UI primitives, and runtime behavior. Treat it as the language authority. The present chapter explains how an editor user encounters the language; it deliberately does not duplicate the reference catalog.

Note. Keep maintained automation as readable source under version control. A macro that edits valuable files deserves the same review discipline as a small program—because that is exactly what it is.

Index

- exit-after-run-macro, 9
- load-recursive, 9
- run-macro, 9

Abort, 25

Acquire dialog, 11

- Load all, 11

Add, 7

Apply compare to original, 38

Apply original to compare, 38

ASCII, 30

ASCII table, 30

AUTOEXEC, 26, 41

Automatic Setup, 38

autosave, 8

background work, 2

backup, 8

basic concepts, 2

Bento hex editor, 35

Bento workspace, 3

binary autodetection, 8

binary detection, 35

binary file, 3

- command line, 10

binary notation, 28

block

- copy, 17

- delete, 17

- indent, 36

- indentation, 17

- inter-window, 17

- marking, 16

- move, 17

- undent, 36

- visibility, 16

Block > Copy, 17

Block > Delete, 14

Block > Indent, 15, 17, 36

Block > Load block, 17

Block > Mark lines, 16

Block > Persistent blocks, 16

Block > Undent, 15, 36

Block > Window copy, 17

Block > Window move, 14

block operations, 16

box-drawing characters, 27

brace matching, 36

breakpoint, 41

build current file, 38

bytes, 3

calculator, 28

- arithmetic, 29

- bitwise operations, 29

- memory, 29

- number base, 28

- paste, 29

- selected text, 29

Cancel, 25

cancellation, 3

case conversion, 17

case-sensitive matching, 34

Center line, 21

character table, 30

clipboard, 13

- character table, 30

Close, 19

code folding, 2, 32, 37

- gutter, 32

- projection, 5, 32

code formatter, 21

code formatting, 38

code point, 30

color setup, 8

color theme, 35

column block, 16

- sorting, 17

- tabs, 15

command line, 9

- run-macro, 41

compiler diagnostics, 38

compiler error

- navigation, 38

compiler profile, 38

- automatic setup, 38

- build hooks, 38

- filename association, 35

compiler warning, 38

Continue, 25

control characters, 30

coprocessor

- telemetry, 3

cursor control, 5

cursor position marker, 2

Cut command, 14

decimal notation, 28

default path, 35

DEFAULT profile, 23

Del, 7

Delete, 7, 17

Delete block, 14

- deleting text, 14
- dense display, 32
- derived state, 37
- desktop, 2
- dialog fields
 - +/-, 28
 - =, 29
 - And, 29
 - Autodetect binary files, 35
 - Base, 28
 - Case sensitive, 24
 - Clr, 28
 - Colortheme, 35
 - Compiler profile, 35
 - Copy, 35
 - CR/LF at EOF, 13
 - Default path, 35
 - Delete, 35
 - Description, 34
 - Extension, 34
 - Global search, 24
 - Keep all files open, 25
 - Mem Rcl, 29
 - Mem Sto, 29
 - New, 35
 - Or, 29
 - Paste, 29
 - Post-load macro, 35
 - Pre-save macro, 35
 - Profile ID, 34
 - Restrict to marked block, 24
 - Restrict to workspace, 25
 - Search all windows, 24
 - Search editor windows, 25
 - Spaces, 15
 - Truncate whitespace, 15
 - Type, 24
 - Use tabs, 15
 - Xor, 29
- display columns, 3
- document, 2
- Double lines, 27
- Edit, 7
- Edit > Append, 13
- Edit > Cut, 14
- Edit > Cut and append, 13
- Edit > Insert, 13
- Edit > Paste, 13
- Edit > Redo, 18
- Edit > Undo, 18
- Emoji table, 30
- End marking, 16
- EOF marker, 13
- external formatter, 21
- F11, 3
- File > Acquire, 11
- File > Export to PDF, 21
- File > Get last, 11
- File > Info, 12, 13
- File > Load, 11
- File > Open, 4, 11
- File > Open journal, 3, 11
- File > Open live log, 3, 11
- File > Open workspace, 12
- File > Revert, 12
- File > Save, 12
- File > Save all, 12
- File > Save As, 12
- file compare, 38
 - difference markers, 32
- file comparison, 22
- file formatting, 21
- File Information dialog, 12
- filename-extension profile, 34
 - matching, 34
- format line, 23
 - tab stops, 23
- formatting
 - scope, 21
- Found List, 25
- Fullscreen, 3
- function keys
 - context-sensitive, 3
 - modifiers, 3
- function-key line, 32
- Get, 20
- Get Last dialog, 11
- Git, 38
 - changes, 38
 - diff, 38
 - status marker, 38
- glob patterns, 9
- Goto line number, 5
- gutter, 32
- Help, 3
 - context-sensitive, 3
 - history, 3
- Help > Keys, 4, 5
- Help > Performance panel, 3
- hex editor, 11
- hexadecimal, 28
- Hide/show block mark, 16
- history limits, 8
- horizontal scrolling, 5
- indent style, 36
- indentation, 36
- indentation-sensitive language, 36
- information projection, 32
- insert mode, 3, 13
- inserting text, 13
- Installation and setup, 34
- installation and setup, 7
- Installation and setup > Color setup, 8
- Installation and setup > Compiler profiles, 38
- Installation and setup > Key mapping, 7

- journal, 3
- key sequence, 7
- keyboard
 - Alt, 3
 - Alt+Enter, 13
 - Alt+F1, 3
 - Alt+Shift+A, 30
 - Alt+Shift+C, 28
 - Alt+Shift+E, 30
 - Backspace, 3
 - Ctrl, 3
 - Ctrl+Enter, 13
 - End, 5
 - Enter, 3, 13
 - Esc, 2, 3, 11, 13, 26
 - Esc Abort, 3
 - F1, 3, 9
 - F10, 3, 4
 - F11, 3
 - Home, 5
 - PageDown, 5
 - PageUp, 5
 - Shift, 3
 - Shift+F1, 3
 - Shift+Tab, 3, 13
 - Super, 3
 - Tab, 3, 13
- keymap, 7
 - macro binding, 40
- keystroke macros, 26
- language detection, 35, 37
- language-specific features, 37
- large files
 - line index, 5
 - undo, 18
- Latin-1, 30
- left margin, 23
- line block, 16
- line drawing, 27
 - double lines, 27
 - mouse, 27
 - rectangle, 27
- line ending, 13
- line numbers, 32
- List, 19
- live log, 11
- Load, 8, 25
- Load all, 11, 25
- Load block, 17
- Load command, 11
- loading files, 11
- log window, 3
- long lines, 5
- Lower case, 17, 21
- Macro > Macro Library, 26, 40
- Macro > Recording start/stop, 26
- macro binding, 26
- macro definition, 40
- macro directory, 26
- macro file, 40
- Macro Library, 26
- macro recording, 26
- MACRO_TO_KEY, 40
- margins
 - profile setting, 35
- Mark columns, 16
- Mark stream, 16
- marker stack, 5
- matching parentheses, 38
- menu line, 3
- message line, 3
- minimap, 2, 32
 - navigation, 5
- modeless macro UI, 40
- mouse, 3
 - terminal support, 6
- Move, 17
- MRMAC, 40
 - compiler, 40
 - execution context, 40
 - runtime UI, 40
 - undo, 18
- MRMAC debugger, 41
- multiple-file replace, 25
 - cancellation, 25
 - Replace All, 25
- multiple-file search, 25
 - cancellation, 25
 - results, 25
- navigation, 5
- New, 7
- Next, 19
- non-breaking space, 30
- Open command, 11
- Other > Build current file, 35, 38
- Other > Git changes, 38
- Other > Installation and setup, 4, 7
- Other > Installation and setup > Filename extensions, 34
- Other > Installation and setup > User interface settings, 32
- Other > Match parenthesis, 5, 38
- Other > Next compiler error, 38
- outline, 5, 37
- overwrite mode, 3, 13
- PageDown, 5
- PageUp, 5
- paragraph formatting, 21
- parenthesis matching, 5
- Paste command, 13
- paths setup, 8
- PCRE2, 24
- PDF export, 21
- Performance panel, 3
- persistent blocks, 16

- plain text, 37
- position marker, 5
- post-load macro, 35
- pre-save macro, 35
- Previous, 19
- Previous compiler error, 38
- progress, 3
 - Acquire, 11
- Re-format code, 21
- Re-format paragraph, 21
- read-only, 3
- recent files, 11
- recording marker, 26
- recovery, 18
- Redo, 18
- redo branch, 18
- regular expression, 24
- remote session, 33
- Ren, 7
- Repeat previous search, 25
- replace, 24
 - confirmation, 25
- Revert, 25
- right margin, 23
- runtime menu, 40
- Save, 8
- Save All command, 12
- Save As, 8
- Save As command, 12
- Save block, 17
- Save block, 17
- Save command, 12
- saving files, 11
- Search, 25
- search, 24
 - literal, 24
 - word, 24
- Search > Get position, 5
- Search > Goto line number, 5
- Search > Multiple file search, 25
- Search > Push position, 5
- Search > Repeat previous search, 25
- Search > Search and replace, 25
- Search > Search for text, 24
- Set right margin, 23
- settings
 - draft, 7
 - validation, 7
- settings.mrmac, 7
- smart indentation, 36
- Snippet SideKick, 13
- spaces
 - indentation, 36
- split view, 19
- startup files, 9
- startup macro, 41
- Static Mode, 3
- status line, 3, 32
- stream block, 16
- syntax highlighting, 37
- systemd journal, 11
- tab expansion, 15
- tab size, 15
- tabs, 15
 - indentation, 36
 - profile setting, 35
- task overview, 2
- terminal font, 31
- terminal size, 33
- Text > File compare, 22, 38
- Text > Hex editor, 10
- Text > Line drawing, 27
- Text > Re-format code, 38
- Text > Set left margin, 23
- Text > Upper case, 17, 21
- theme, 8
- Tile, 19
- Tools > ASCII table, 30
- Tools > Calculator, 28
- Tools > Emoji table, 30
- Turn marking off, 16
- TVision, 2
- Undent, 17
- Undo, 18
- undo history
 - operation boundary, 18
- Unicode, 13, 30
- Unlink, 19
- Use, 7
- user-interface settings, 8
- UTF-8, 3
 - input, 13
- viewport
 - minimap navigation, 32
- virtual desktop
 - navigation, 19
- virtual desktops, 19
- virtual space, 15
 - materialization, 15
- visual columns, 36
- whitespace policy, 21
- window, 2
 - cascade, 19
 - frame markers, 2
 - hidden, 19
 - hide, 32
 - macro link, 40
 - minimize, 32
 - minimized, 19
 - tile, 19
 - zoom, 32
 - zoomed, 19
- Window > Hide, 32
- Window > Link, 19
- Window > List, 4

- Window > Minimize/restore, 32
- Window > Open, 19
- Window > Organize > Cascade, 19
- Window > Zoom, 32
- Window copy, 14
- Window copy block, 14
- window link, 19
- Window List dialog, 19
- window management, 19
- Window move, 17
- Window move block, 14
- word delimiters, 5
- workspace, 12
 - autoload, 12
 - autosave, 12
 - main file, 20